

Snap Shop

Project Team

Zaraar Malik 21I-2705
Ahmed Kamal 21I-0308
Raffay Khan 21I-0335

Session 2021 - 2025

Supervised by

Dr. Hammad Majeed

Co-Supervised by

Muhammad Farrukh Bashir



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2024

Contents

1	Anti Plagiarism Declaration	1
2	Abstract	2
3	Introduction	3
3.1	Project Vision	3
3.1.1	Problem Statement	4
3.1.2	Project Scope	4
3.1.3	Project Objectives	4
3.1.4	Project Constraints	5
3.1.5	Business Opportunity	5
3.2	Current Technology and Comparison	6
3.2.1	Research on Similar Technology	6
3.2.1.1	mirrar.com	6
3.2.1.2	Diffusion Try Ons[4]	7
3.2.1.3	ZERO 10[6]	7
3.2.1.4	Pulpoar[3]	8
3.2.2	Comparison with Snap Shop	8
4	Stakeholders and User Description	9
4.1	Non-User Stakeholders	9
4.2	User Summary	10
4.3	User Environment	10
5	Product Features	11
5.1	Modules	11
5.1.1	Custom Image Generation	11
5.1.2	Customer Personalization	11
5.1.3	Ad Generation	12
6	Software Requirements Specification	13
6.1	Functional Requirements	13
6.2	Non-Functional Requirements	14

7	Project timeline	15
7.1	TimeLine	15
8	First Iteration	16
8.1	Data Pre-processing	16
8.2	Figma Design	16
8.3	Prototypes	17
8.4	Feedback	17
8.5	Single Item Image Generation	17
9	System Overview	18
9.1	Architectural Design	19
9.1.1	Components	19
9.1.2	Connectors	20
9.1.3	Data Elements	20
9.1.3.1	Input Data	20
9.1.3.2	Intermediate Data	20
9.1.3.3	Output Data	20
9.2	Design Models	20
9.2.1	Activity Diagram	21
9.2.2	Data Flow Diagram	22
9.2.3	System Sequence Diagram	23
9.2.3.1	Single Item generation	23
9.2.3.2	Multiple Item generation	24
9.2.3.3	Single Item generation with custom Model	25
9.2.3.4	Multiple Item generation with custom Model	26
9.2.4	State Transition Diagram	27
10	Project Use Cases	28
10.1	Use Cases	28
10.2	Use Case Diagram	31
11	Project Testing	32
11.1	Testing Techniques	32
11.1.1	Static Testing Techniques	32
11.1.1.1	Test Case 1: Peer Reviews	32
11.1.1.2	Test Case 2: Walkthrough	33
11.1.1.3	Test Case 3: Code Inspection	33
11.1.2	Dynamic Testing Techniques	33
11.1.2.1	Test Case 1: Unit Testing (Structure-based)	34
11.1.2.2	Test Case 2: Integration Testing (State Transition Testing)	34

11.1.2.3	Test Case 3: System Testing	34
11.1.3	Experience-Based Testing Techniques	35
11.1.3.1	Test Case 1: Error Guessing	35
11.1.3.2	Test Case 2: Exploratory Testing	35
11.1.3.3	Test Case 3: Use Case-based Testing	36
11.1.4	Specification-Based Testing Techniques	36
11.1.4.1	Test Case 1: Equivalence Partitioning	36
11.1.4.2	Test Case 2: Boundary Value Analysis (BVA)	37
11.1.4.3	Test Case 3: Decision Table Testing	37
11.2	Conclusion	37
12	Project Maintenance	38
12.1	Continuous Integration and Continuous Development	38
12.2	Maintenance Test	39
13	Conclusion	40
	References	41

List of Figures

9.1	Architecture of the system	19
9.2	Activity Diagram	21
9.3	Data Flow Diagram	22
9.4	Single Item Generation	23
9.5	Multiple Item generation	24
9.6	Single Item generation with custom Model	25
9.7	Multiple Item generation with custom Model	26
9.8	State Transition Diagram	27

List of Tables

3.1	Comparison of Snap Shop with current technology	8
4.1	User Summary	10
7.1	Timeline for the Project	15

Chapter 1

Anti Plagiarism Declaration

This is to declare that the above FYP report produced under the following project

Title : Snap Shop

is the sole contribution of the author(s) and no part hereof has been reproduced on as it is basis (cut and paste) which can be considered as Plagiarism. All referenced parts have been used to argue the idea and have been cited properly. I/We will be responsible and liable for any consequence if violation of this declaration is determined.

Student 1

Name : Zaraar Malik

Signature : ZARAAR MALIK

Student 2

Name : Ahmed Kamal

Signature : AHMED KAMAL

Student 3

Name : Raffay Khan

Signature : RAFFAY KHAN

Co-Supervisor

Name : Farrukh Bashir

Supervisor

Name : Dr. Hammad Majeed

Chapter 2

Abstract

The Fashion Industry is a Multi-Billion Dollar Industry[2] dedicated to the production and sales of multiple clothing items all over the world. This Industry has made quite an impact on the society. Individuals, all over the world, from different age groups have their specific likes and dislikes about their clothes and their brands. However, with so many booming Brands, the customers are compelled to try something new but they always have a doubt in their mind "Is this worth a shot?". "Will I really look good in this ?". "I wish I could try multiple dress suits quickly but alas I would have to wear each suit one at a time" This has left the Brands with a question "How to draw in the unattainable ?". "How to provide a completely new experience that will blow away their minds". **SnapShop** helps in not only enhancing the marketing strategy of new and emerging Brands but also provide an enriching experience for the customers which will help them try different and new clothing items. It allows the customers to wear multiple clothes at the same time so that they themselves can design an outfit of their own choice without spending a lot of time. Moreover, it also allows the customers to view themselves wearing those particular clothes in different poses and environments which further enhances the customer experience. Furthermore, it also allows fashion designers to generate video ads for their particular designed clothes.

In conclusion, the main goal of Snap Shop is to Revolutionize the Fashion Industry, Streamline the process of ad generation from the production's point of view and also enhance customer experience by utilizing the power of Artificial Intelligence.

Chapter 3

Introduction

Snap Shop is a Web Based application which aims to revolutionize the Fashion Industry with the help of Artificial Intelligence. The project aims to target 2 classes which is mentioned as following :

1 - The Customers (more particularly The General Public and Fashion Enthusiasts).

2 - The designers of fashion items (handbags,shirts,shoes,ties,coats etc)

This product will provide a simple yet innovative way to wear multiple clothes without actually wearing them in real life. This will help the customers to have a better idea of how they will look in different clothes. This will also save tremendous amount of time due to the lack of trying rooms in different outlets of different brands. Moreover, it will provide a platform for not only the fashion designers but generally anyone who has designed a fashion item and wants to market it without facing the hassle of Media Team, Fashion Events, Fashion Models. It will allow them to generate model images with their particular clothing item(s) along with video ad generation as well.

3.1 Project Vision

To streamline the process of ad generation for new and emerging Fashion companies. Provide an enhanced and never-seen-before customer experience with the help of AI that will truly change purchasing of fashion items. To bypass the long and time-consuming human effort to generate fashion ads and provide a 1-click solution with little to no cost in terms of money and time. Allow the customers to create their own personalized outfit by giving them the power to wear their desired clothing item without actually wearing it. Moreover, to also allow the customers to wear more than one item at the same time to design a truly unique outfit for a particular event or occasion.

3.1.1 Problem Statement

Over the past few years, the fashion industry has been growing quite steadily. With the increase new brands, new companies, new designs etc, the customer are quite confused and intrigued at the same to try new brands. However, if we look from the developers point of view, the increase in saturation has not only caused an increase in the marketing price but has also given birth to new and modern ways to market a product. Participating in different Fashion events, Hiring Models in Fashion Events to wear one's desired clothes, Hiring a media team to develop an entire personal album for one's product can be very costly and time consuming and if the results are not up to the mark, an additional effort is then executed to refine the obtained results. This can put a lot of pressure on emerging brands which have spend an enormous amount of effort, expense and dedication in developing a product. From a customers point of view, loyal customers will find it quite difficult even to try new brands. Moreover, trying clothes one after the other can also be quite time consuming and some customers might avoid trying new brands just because of the time factor. What if a customer wants to make an outfit of his/her own choice, several clothes would have to be worn in order to get the right outfit which can further increase the time factor leaving an unsatisfied feeling in the customer.

3.1.2 Project Scope

This project aims to provide a web based application for the users. The web application will provide the users to upload their desired clothing item image or images along with their own images. They will also have an option not to upload their own images. The web application will also have a database which will be unique for each user as it will allow them to store their custom designed outfits using the AI model provided in the web application. Moreover, their will more flexible options for the users such as pose-generation, custom environment and finally video ad generation. This video ad generation is particularly for the producers of fashion items however it can be utilized by the customers as well.

3.1.3 Project Objectives

1. **Web Application** Create a Web application to facilitate users in their day to day shopping and also facilitate designers in generating new ads for their newly designed products.
2. **Custom Fashion Item** : Allow the users to select a desired clothing item and tell the application to generate an image using the specified fashion item.

3. **Multiple Fashion Items :** Allow the users to select multiple desired clothing items and tell the application to generate an image using the specified fashion items.
4. **Pose Generation:** Allow the users to select their desired pose in which they want the image to be generated
5. **Personalization** Allow the users to combine the first 2 steps with their own images and map their desired images onto their own images. This will enhance the user experience
6. **Custom Environment** Allow the users to combine all of the above mentioned steps with a text prompt which specifies the environment as models such as text to image models in which a custom environment is specified.
7. **Video Ad Generation** Allow the users to generate a small 5-10 second video for their specified clothing items

3.1.4 Project Constraints

1. **Real Time Inference :** To provide the users with a real time output for cloth item mapping on either their personal images or pre-defined images
2. **Model Selection and Training :** Model selection and training will be a crucial part of the project as it will determine the inference speed and also cost of deployment with other crucial items.
3. **User Friendly Application** The Web Application must in such a way that will allow the users to easily navigate and use it as per their requirement.
4. **Data Base Integration** The database is also a crucial part of the project which will be used to maintain the data of each User as the website is being used.
5. **Multiple Users** This will be a major factor in which if more than 1 users is utilizing the website's model at the same time, how to tackle this issue use a serial approach or parallel approach.? This will highly depend on out deployment strategy.

3.1.5 Business Opportunity

The business opportunity this project holds is quite massive. It bypasses the process of Hiring Models, Hiring a Media Team, Participating in Fashion Events, Multiple Models for Multiple Different Categories of products. Rather, this project just provides a simple yet powerful interface that will practically allow anyone with a new clothing item or multiple new clothing items to generate ads via uploading their product images on the

website, writing a brief text prompt and explaining how they want the generated ads to look like. This includes the pose of the character that will be in the generated image and the environment of the entire image. So, this product has huge potential for Fashion Industry from the developer's perspective. Moving over to the customer's perspective, everyone would love to utilize this app since the biggest problem is trying rooms in brand outlets. Barely 1 or 2 trying rooms are available and during rush hours, they are mostly occupied. However, with the help of Snap Shop, the customers can easily see themselves in their specified clothes without wearing them in real life. This will not only save them the time but it will also boost sales and every brand would want to get their hands on this product.

3.2 Current Technology and Comparison

Currently, there is no application that provides such a solution as proposed by us. However, there are different applications that provide some of the features as mentioned in our project. For example Video Ad Generation is provided by 1 application and Cloth Swapping is provided by another application and such like this. Following are some details of these applications and technology

3.2.1 Research on Similar Technology

3.2.1.1 mirrar.com

This has a web application[1] and a mobile application to enhance the experience of the users. It utilizes the power of Augmented Reality. It access's the camera of the user's mobile phone and while recording the video of the user, the user can select different fashion items which they would want to wear and show the user how he/she would look like in that fashion item.

Limitations

1. **No Custom Environment** Users cannot change their environment, it just maps on a small item on their body
2. **No database Integration** Users cannot store or keep a record of their own designed outfits
3. **Pre-defined Clothing items** The application is able to map only those clothing/fashion items that are pre-defined in the application. Users cannot use an image of their own choice.

3.2.1.2 Diffusion Try Ons[4]

This is not a a web application but a technology that utilize a very advance part of Artificial Intelligence which is called Stable Diffusion. These are models which generate images from noise with a process which takes into account a random noise vector from a specified distribution and try to de-noise this in several number of iterations until an image is formed.

Limitations

1. **No Web Application** No web application means general public cannot easily use this.
2. **No Custom Environment** Users cannot change their environment, it just maps on a small item on their body.
3. **No database Integration** Users cannot store or keep a record of their own designed outfits
4. **Limited Clothing Items** Though the users can upload their own desired clothing item however its is restricted to just Full-Body or Half-Body Clothing items.
5. **Multiple Clothing Items** It does not allow the user to upload load multiple clothing items to be mapped onto their bodies. It just handles 1 clothing item

3.2.1.3 ZERO 10[6]

This has a mobile application to enhance the experience of the users. It utilizes the power of Augmented Reality. It access's the camera of the user's mobile phone and while recording the video of the user, the user can select different fashion items which they would want to wear and show the user how he/she would look like in that fashion item.

Limitations

1. **No Custom Environment** Users cannot change their environment, it just maps on a small item on their body.
2. **No database Integration** Users cannot store or keep a record of their own designed outfits
3. **Limited Clothing Items** Though the users can upload their own desired clothing item however its is restricted to just Full-Body or Half-Body Clothing items.
4. **Multiple Clothing Items** It does not allow the user to upload load multiple clothing items to be mapped onto their bodies. It just handles 1 clothing item

5. **Poor Image Quality** The resultant image looks as if a clothing item image was just copy pasted on the users body Rather than actually "making them wear it".

3.2.1.4 Pulpoar[3]

This is a web application that allows its user to perform facial make-up, change their hair color, polish their nails, run a health and skin diagnostics checkup. The techniques used by their product can transition from hair color to body clothes and from nail polish to handbags and wrist watches and rings. However, this is a paid application which further leaves an unpleasant feeling amongst the customers.

Limitations

1. **No Custom Environment** Users cannot change their environment, it just maps on a small item on their body.
2. **No Clothing Items** Though the users cannot upload their own desired clothing item images.
3. **Multiple Clothing Items** It does not allow the user to upload load multiple clothing items to be mapped onto their bodies.

3.2.2 Comparison with Snap Shop

The above mentioned technologies and applications are just a few out of many however, we can make a basic ideas as to how our product differs from the current available technology. Following is a comparison table .

Item	Custom Image	Multiple Custom Images	Personalization
Snap Shop	Yes	Yes	Yes
mirrar	No	No	Yes
Diffusion Try On	Yes	No	Yes
ZERO 10	No	No	Yes
Pulpoar	No	No	No

Item	Custom Environment	Video Ad Generation	Pose Generation
Snap Shop	Yes	Yes	Yes
mirrar	No	No	No
Diffusion Try On	No	No	Yes
ZERO 10	No	No	No
Pulpoar	No	No	No

Table 3.1: Comparison of Snap Shop with current technology

Chapter 4

Stakeholders and User Description

4.1 Non-User Stakeholders

The Non-User Stakeholders is basically the entire team. The Team Members are mentioned as following :

1. **Zaraar Malik** The main responsibility of this individual to perform the implementation of the entire project. Work on optimization techniques and improve the inference of the models being used in the project. Moreover, he also has to work on model deployment part.
2. **Raffay Khan** The main responsibility of this individual is to work on the website of the project along with designing the website on Figma. Moreover, he also has to provide assistance to the development part of the project in training and testing models
3. **Ahmed Kamal** The main responsibility of this individual is to work on the maintenance phase of the project, as well as improving the image and video generation models. Make a CI/CD pipeline for the project, ensure each module is working properly, make new test cases after a new module has been merged into the project.

This also includes the Co-Supervisor and Supervisor which are mentioned below:

1. **Co-Supervisors: Farrukh Bashir** Sir Farukh Bashir is responsible for providing assistance in small problems related to the project.
2. **Supervisor: Dr. Hammad Majeed** He is the supervisor of the project and clarifies any major problem or issue in the project.

4.2 User Summary

max width=		
Name	Description	Responsibility
Fashion Enthusiast	This is an individual of the society that loves to do shopping in a daily or weekly basis. This individual has a passion for new styles, designs and wants to make new outfits from one's own imagination	Feedback on fashion items, Contribute Style Preferences ,Engage with Fashion Content
Fashion Designers	This individual of the society is responsible for making new clothing designs and new fashion items such as handbags, wrist watches, rings, hats etc.	Create Custom Designs , Provide Design Concepts,Refine User Personalization
General Public	This is an individual of the society that does not actively take part in purchasing fashion items but occasionally buys items.	Provide Consumer Insights, Participate in Marketing Campaigns, Drive Market Demand

Table 4.1: User Summary

4.3 User Environment

The users of Snap Shop can access the platform on the internet. The users will be able to upload their own desired fashion item images along with their own images or leave it empty. Then the application will generate their desired images. This platform will also be available in brand outlets to increase the pace of trying different item while shopping rather than wearing each item 1 at a time.

Chapter 5

Product Features

Identify the product or application whose requirements are specified in this document. Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. [?].

5.1 Modules

5.1.1 Custom Image Generation

Image generation of a human model in specified pose[5] and environment wearing given fashion items in specified orientations such as different color, zipped up or down, sleeves rolled or not, etc.

1. **Desired Set of Images:** This will allow the user to select a single or multiple fashion item images at a time and it will generate an image of a human model wearing those selected items.
2. **Pose Generation:** This will allow the user to select a desired image pose and it will generate an image based on the selected pose.
3. **Custom Environment:** Customized prompt can also include a change in environment and will generate an image based on the prompt

5.1.2 Customer Personalization

Image generation of a custom human model defined by the user them self with all the functionalities of the first module namely multiple fashion items, custom pose, custom

environment, etc. molded to fit custom human model whilst the body proportions remains intact.

1. **Single Item Image Customization:** Allow the customers to upload their image and select a desired clothing item and an image will be generated of the customer wearing that item.
2. **Multiple Item Image Customization:** Allow the customers to upload their image and select multiple desired clothing items and an image will be generated of the customer wearing that item.
3. **Image Customization with Pose Generation:** Allow the customers to upload their image and select multiple desired clothing items and also their desired pose and an image will be generated of the customer wearing that item with the desired pose.

5.1.3 Ad Generation

Generation of a short video of 5-8 seconds with custom human model, and custom fashion items, pose and generation

1. **Video Ad Generation:** This will allow the producers of different fashion products to generate a small 5-10 second ad about their fashion product. This video has all the previously mentioned features. After generation the desired image, the producer can further generate a small video of that image.
2. **Story Generation:** This can understood as each product has some uniqueness and charisma. The generated video will also have a small story line which will uniquely define the product being advertised.

Chapter 6

Software Requirements Specification

Identify the product or application whose requirements are specified in this document. Describe the different types of reader that the document is intended for, such as developers, project managers, marketing staff, users, testers, and documentation writers. [?].

6.1 Functional Requirements

1. User Authentication

- Users must be able to create an account and log in using email and password.
- Users must be able to reset their passwords via email verification.

2. Image Upload

- Users must be able to upload images of fashion items in supported formats (e.g., JPG, PNG).
- Users must be able to upload their own images (full-body or portrait) for the multi-garment mapping feature.

3. Multi-Garment Image Mapping

- The system must generate images showing the user wearing the uploaded fashion items.

4. Pose Generation

- Users must be able to select from a predefined list of poses for their new image.

- The application should support the generation of custom poses based on user input(a reference image having a different pose).

5. Video Ad Generation

- Users must be able to create video advertisements featuring the multi-garment mapping.
- users are able to briefly describe the features of the video

6. Real-Time Preview

- The application must provide a real-time preview of the generated images and videos before finalizing.

7. User Profile Management

- Users must be able to view and edit their profiles, including their uploaded images and generated content.

6.2 Non-Functional Requirements

1. Performance

- The application should generate images within reasonable amount of time after uploading.
- The system should support at multiple users at a time

2. Usability

- The user interface must be intuitive and easy to navigate, allowing users to upload images and access features with minimal training.
- The design should be responsive, ensuring a seamless experience.

3. Scalability

- The application must be scalable to accommodate increasing user loads and image/video processing demands.

4. Maintainability

- The codebase should follow best practices for maintainability, allowing for easy updates and feature enhancements.

Chapter 7

Project timeline

7.1 TimeLine

Iteration #	Time Frame	Tasks/Modules
01	Sept - Oct (2024)	Data Pre-processing, Figma Design, Prototypes Feedback, Single Item Image Generation
02	Oct - Nov (2024)	Multiple Item Image Generation
03	Nov - Dec (2024)	Pose Generation, Custom Environment
04	Dec - Jan (2024, 2025)	Personalization (Custom Fashion Item + Custom Image)
05	Feb - Mar (2025)	Simple Video Ad Generation
06	Mar - Apr (2025)	Personalized Video Ad Generation
07	Apr - May (2025)	Website Development, User Interface
07	May - Jun (2025)	Deployment, Runtime Inference

Table 7.1: Timeline for the Project

Chapter 8

First Iteration

The first iteration of the SnapShop project was completed by early-October 2024, focusing on initial stages of the project development.

8.1 Data Pre-processing

In this early phase, we selected a pre-trained generative model, which will form the basis for SnapShop. We are now working on fine-tuning this model by collecting data on multiple types of wearable items such as shirts, shoes, watches, jewelry, and more. The data collection process involves gathering images of diverse fashion items from multiple different perspectives, ensuring a large and comprehensive dataset. This dataset will be pre-processed, including pre-processing techniques like resizing, normalization, and categorization, in order to make the dataset compliant for the model that we selected.

8.2 Figma Design

During this iteration, we used Figma to create a user friendly interface for the SnapShop platform. The Figma design is mainly for the developer to see how the SnapShop platform will incorporate all its features into the frontend, such as: the ability for users to upload wearable items, choose models, and view the AI generated ads. These designs will be the basis for the final UI/UX implementation for the SnapShop platform, guaranteeing an intuitive user experience.

8.3 Prototypes

We made several prototypes according to the initial designs. These prototypes focus on the flow of uploading a single wearable item, selecting a model, and viewing the image and video ad generation results. The prototypes were thoroughly reviewed and tested by our own FYP team members, including our supervisor and co-supervisor. They were also reviewed by our close friends for constructive criticism. These aforementioned individuals gave the starting feedback on the functionality, user interface, and the user experience.

8.4 Feedback

Feedback was mainly collected from our team members, supervisor, co-supervisor, and close friends. This feedback helped us amend the flawed designs, improve functionality, and address problems related to the user experience of the SnapShop platform. Adjustments were made based on this internal testing, especially focusing on enhancing the flow of the single item image generation process and ensuring that the generated images appeared realistic and properly mapped to the human model.

8.5 Single Item Image Generation

The core functionality developed during this iteration was enabling the user to upload a single wearable item, which is then mapped onto a human model. The fine-tuned model processes the uploaded item and generates an image of a person wearing that item. The focus at this stage was ensuring the generated images were accurate in terms of fit, texture, and overall appearance.

Chapter 9

System Overview

Our choice of Architecture is Component-Based Architecture. Component-Based Architecture (CBA) in system design refers to a methodology where software is built by assembling pre-defined, reusable components. Each component encapsulates a specific piece of functionality or behavior, with well-defined interfaces that govern how components interact with each other. This approach promotes modularity, flexibility, and reusability in software development.

- By separating concerns into distinct components, CBA enables easier maintenance, scalability, and collaboration among developers.
- It's particularly beneficial in large-scale applications where different parts of the system need to be developed, tested, and updated independently while ensuring consistency and efficiency across the entire software ecosystem.

Component-Based Architecture (CBA) offers several benefits in system design:

- **Reusability:** Components can be reused in different parts of the application or even in different projects, saving development time and effort.
- **Modularity:** Each component is self-contained with well-defined interfaces, making it easier to understand, develop, and maintain.
- **Scalability:** Components can be independently scaled based on demand, allowing for more efficient resource utilization.
- **Flexibility:** Components can be replaced or upgraded without affecting other parts of the system, promoting agile development and adaptation to changing requirements.

9.1 Architectural Design

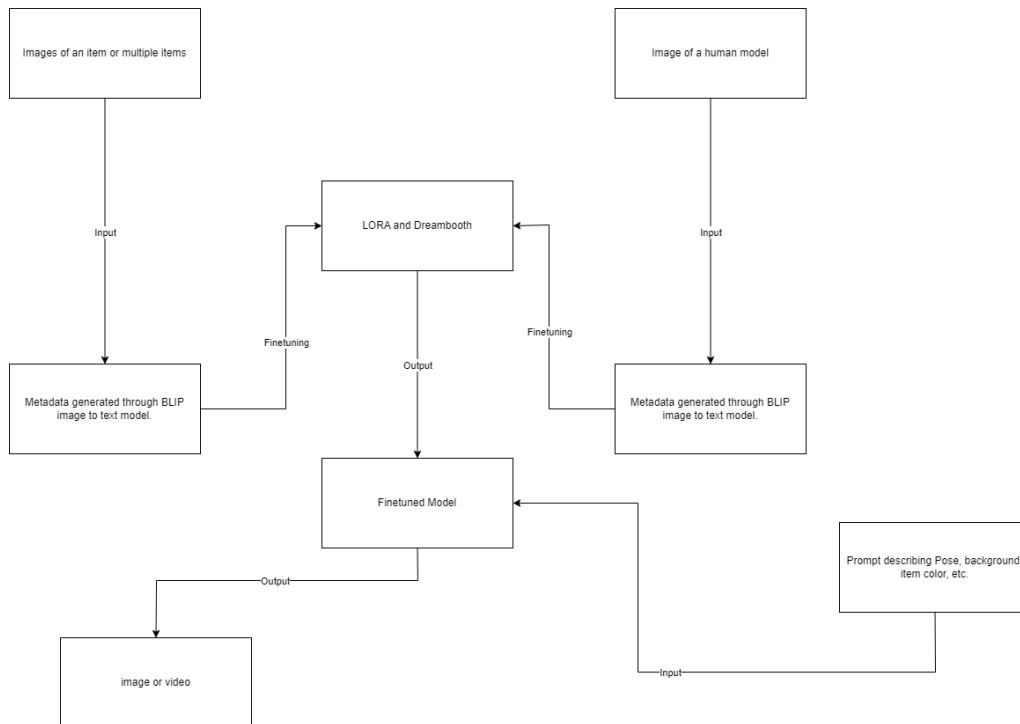


Figure 9.1: Architecture of the system

This system architecture facilitates the generation of image or video content based on input images of items and a human model. The flow includes metadata generation, fine-tuning through LoRA and Dreambooth, and final media output. The system is structured with various components, data flows, and intermediate data.

9.1.1 Components

- **Images of an Item or Multiple Items:** Input images of items that will be passed to the metadata generation process.
- **Image of a Human Model:** Input image of a human model that will also be used in metadata generation.
- **Metadata Generation:** A BLIP image-to-text model generates metadata that describes both the items and human model images.
- **LoRA and Dreambooth:** The metadata generated is used for fine-tuning custom image data in LoRA and Dreambooth models.
- **Fine-tuned Model:** The model, fine-tuned with LoRA and Dreambooth, can now generate custom image or video content.

- **Image or Video (Final Output):** The final output generated from the fine-tuned model based on the inputs.

9.1.2 Connectors

- **Image to Metadata Generation:** Both the images of items and the human model are passed to the BLIP model for metadata generation.
- **Metadata to LoRA/Dreambooth:** The generated metadata is utilized for fine-tuning the model using LoRA and Dreambooth techniques.
- **Fine-tuning to Fine-tuned Model:** The model is updated with the new fine-tuned weights and configurations.
- **Fine-tuned Model to Image/Video Output:** The fine-tuned model is used to generate new image or video content based on the original input.

9.1.3 Data Elements

9.1.3.1 Input Data

- **Images:** Images of items and a human model.
- **Prompt:** Descriptive input for the desired color, pose, and background.

9.1.3.2 Intermediate Data

- **Metadata:** Metadata generated by the BLIP image-to-text model.
- **Fine-tuned Weights:** The configurations and weights updated by LoRA and Dreambooth during the fine-tuning process.

9.1.3.3 Output Data

- **Generated Content:** The final image or video content generated after fine-tuning.

9.2 Design Models

Create design models as are applicable to your system. Provide detailed descriptions with each of the models that you add. Also ensure visibility of all diagrams.

9.2.1 Activity Diagram

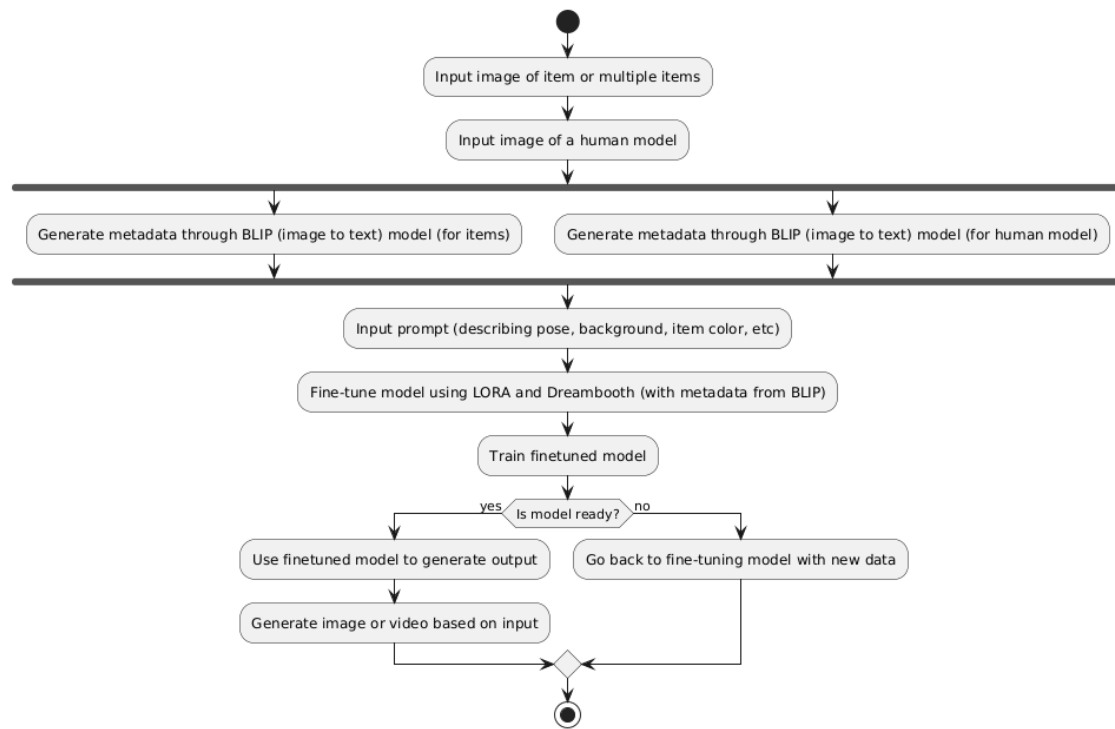


Figure 9.2: Activity Diagram

9.2.2 Data Flow Diagram



Figure 9.3: Data Flow Diagram

9.2.3 System Sequence Diagram

9.2.3.1 Single Item generation

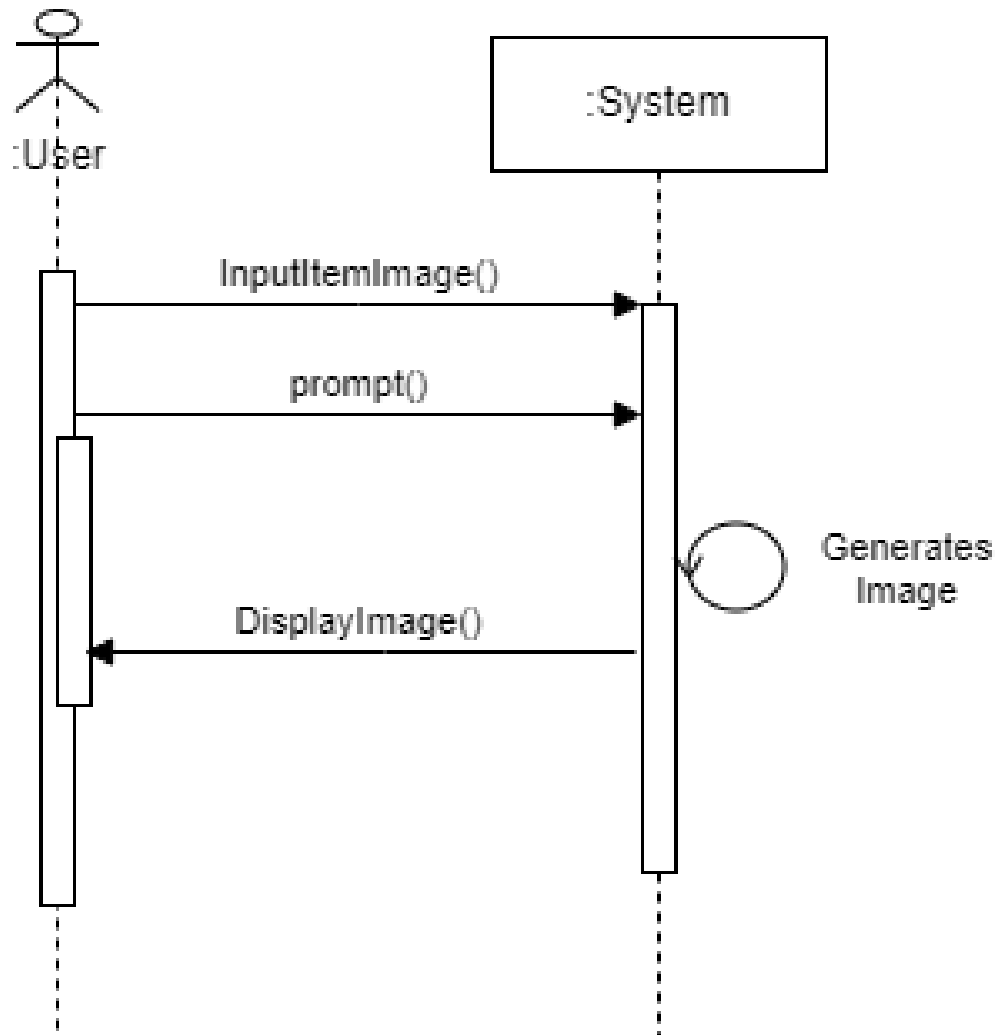


Figure 9.4: Single Item Generation

9.2.3.2 Multiple Item generation

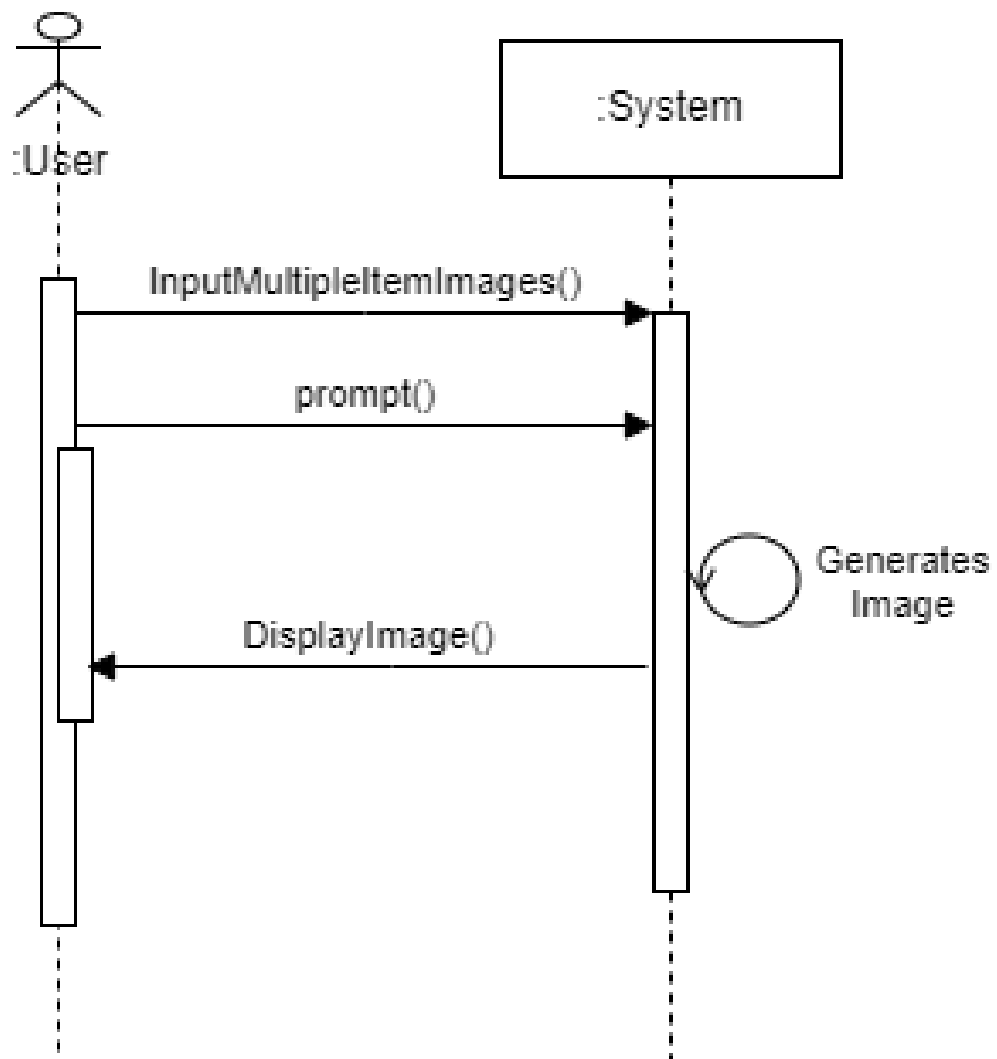


Figure 9.5: Multiple Item generation

9.2.3.3 Single Item generation with custom Model

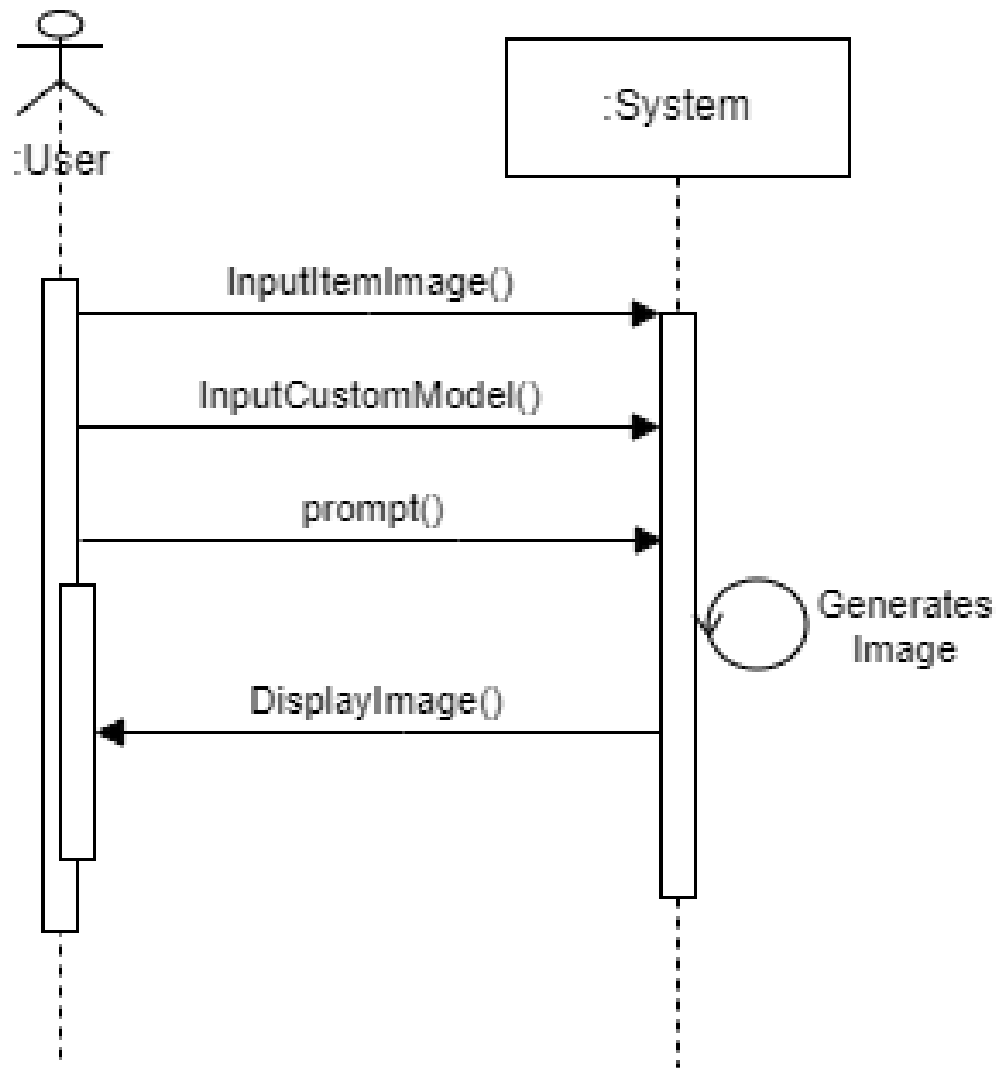


Figure 9.6: Single Item generation with custom Model

9.2.3.4 Multiple Item generation with custom Model

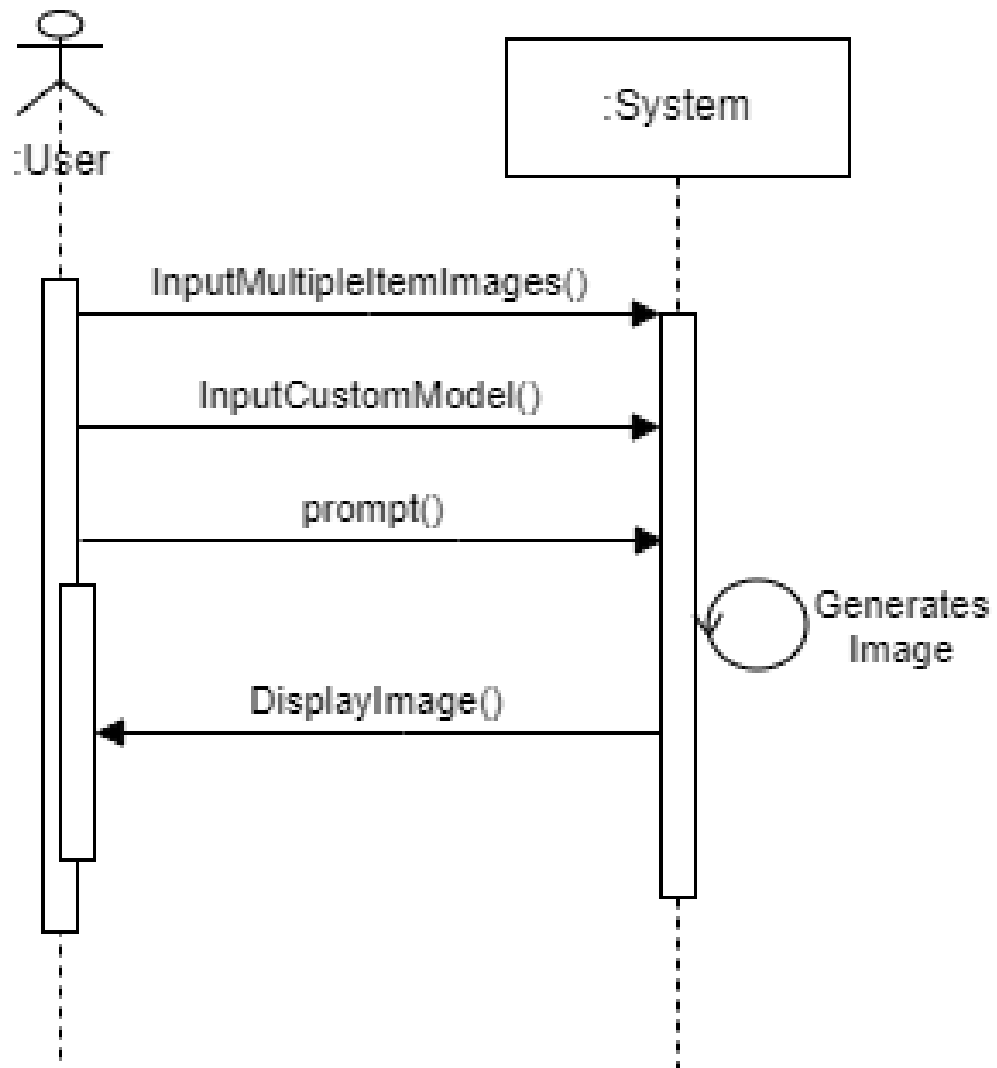


Figure 9.7: Multiple Item generation with custom Model

9.2.4 State Transition Diagram

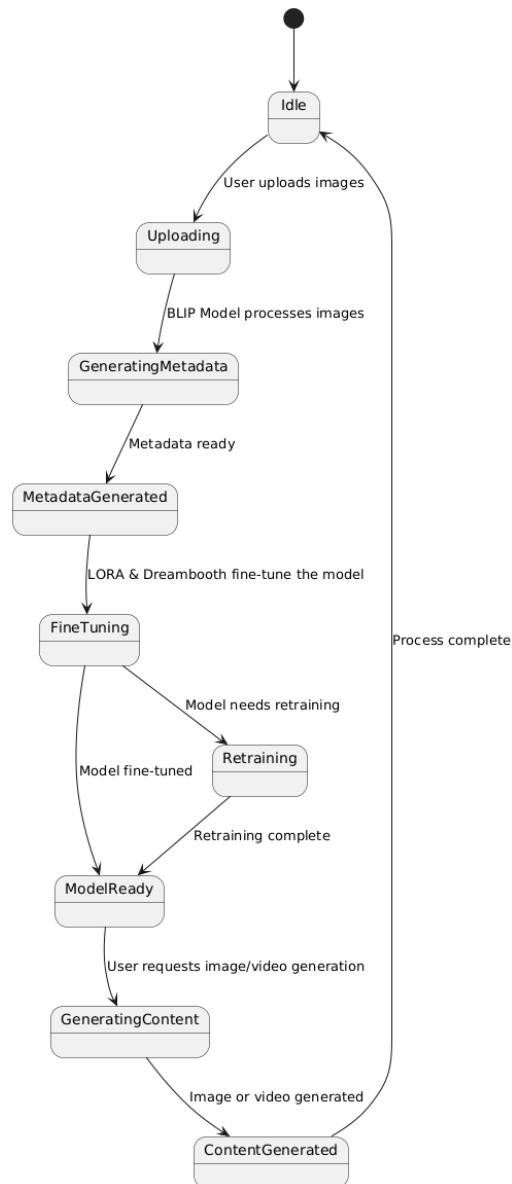


Figure 9.8: State Transition Diagram

Chapter 10

Project Use Cases

10.1 Use Cases

Use Case ID	UC-001
Use Case Name	Image Generation of a Certain Item
Scope	System
Level	User Goal
Primary Actor	User
Stakeholders	User, System
Pre-conditions	User has uploaded an image of the item.
Post-conditions	Image of the item is generated successfully.
Success Scenario	1. User uploads the item image. 2. The system processes the image and generates metadata. 3. The system uses the metadata to generate the image of the item. 4. The image is returned to the user.

Use Case ID	UC-002
Use Case Name	Image Generation of a Certain Item in Specific Pose
Scope	System
Level	User Goal
Primary Actor	User
Stakeholders	User, System

Pre-conditions	User has uploaded an image of the item and has provided a pose description.
Post-conditions	Image of the item in the specific pose is generated.
Success Scenario	1. User uploads the item image and specifies the desired pose. 2. The system processes the image and the pose input. 3. The system generates the item image in the specified pose. 4. The image is returned to the user.

Use Case ID	UC-003
Use Case Name	Image Generation of a Certain Item in Specific Background
Scope	System
Level	User Goal
Primary Actor	User
Stakeholders	User, System
Pre-conditions	User has uploaded an image of the item and has specified the background details.
Post-conditions	Image of the item in the specific background is generated.
Success Scenario	1. User uploads the item image and specifies the background. 2. The system processes the image and the background description. 3. The system generates the item image with the specified background. 4. The image is returned to the user.

Use Case ID	UC-004
Use Case Name	Image Generation of a Certain Item with Specified Color and Orientation
Scope	System
Level	User Goal
Primary Actor	User
Stakeholders	User, System

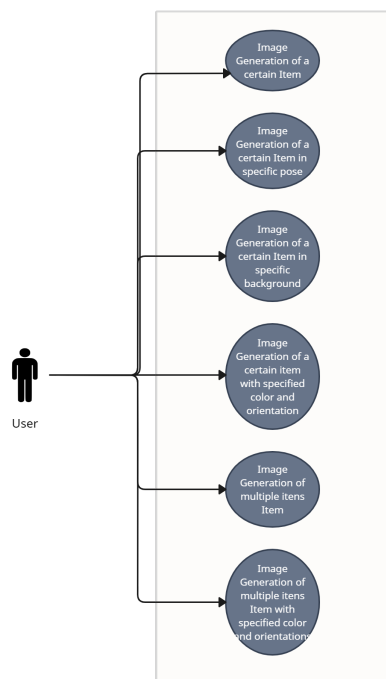
Pre-conditions	User has uploaded an image of the item and has specified the color and orientation.
Post-conditions	Image of the item with specified color and orientation is generated.
Success Scenario	1. User uploads the item image and specifies the color and orientation. 2. The system processes the image and the user inputs. 3. The system generates the item image with the specified color and orientation. 4. The image is returned to the user.

Use Case ID	UC-005
Use Case Name	Image Generation of Multiple Items
Scope	System
Level	User Goal
Primary Actor	User
Stakeholders	User, System
Pre-conditions	User has uploaded images of multiple items.
Post-conditions	Image of all items is generated.
Success Scenario	1. User uploads multiple item images. 2. The system processes all the item images. 3. The system generates the image containing all items. 4. The image is returned to the user.

Use Case ID	UC-006
Use Case Name	Image Generation of Multiple Items with Specified Color and Orientations
Scope	System
Level	User Goal
Primary Actor	User
Stakeholders	User, System
Pre-conditions	User has uploaded images of multiple items and specified their colors and orientations.

Post-conditions	Image of all items with specified color and orientations is generated.
Success Scenario	1. User uploads multiple item images and specifies the color and orientation for each item. 2. The system processes the images and the user inputs. 3. The system generates the image of all items with the specified colors and orientations. 4. The image is returned to the user.

10.2 Use Case Diagram



Chapter 11

Project Testing

11.1 Testing Techniques

11.1.1 Static Testing Techniques

Static testing involves examining project artifacts without actually running the code. It is typically used to catch bugs early in the development process by examining documentation, code, and designs. In this project, static testing helped identify issues during peer review, walkthroughs, and analysis.

11.1.1.1 Test Case 1: Peer Reviews

Objective: Review design documents and code for flaws in logic and structure.

Steps:

1. Gather the team and distribute the design documents.
2. Review the code for inefficiencies and design flaws.
3. Discuss feedback with the team.
4. Make note of areas needing improvement or further clarification.

Expected Outcome: The identification of at least five areas of optimization in the code and documentation.

Actual Outcome: Several inefficiencies were found, particularly in loop structures and redundant logic, leading to code optimizations.

11.1.1.2 Test Case 2: Walkthrough

Objective: Provide the team with an in-depth explanation of the system flow and architecture.

Steps:

1. Schedule a walkthrough session with the project lead.
2. Review the system's design and code with the team.
3. Encourage team members to ask questions and provide feedback.
4. Update the documentation based on feedback.

Expected Outcome: Team gains a clear understanding of system logic, identifying any potential issues.

Actual Outcome: Minor documentation errors were noted and corrected, and the team better understood system functionality.

11.1.1.3 Test Case 3: Code Inspection

Objective: Identify bugs and compliance issues with coding standards.

Steps:

1. Review the code in a formal inspection meeting.
2. Check for compliance with coding standards.
3. Log any bugs or issues found during the inspection.
4. Track the resolution of identified bugs.

Expected Outcome: All major and minor bugs are logged and tracked for resolution.

Actual Outcome: Three major and seven minor bugs were identified and resolved.

11.1.2 Dynamic Testing Techniques

Dynamic testing is performed during code execution to ensure that the system performs as expected in the real world. Such testing is important for catching functionality, logic, and performance-related bugs during runtime.

11.1.2.1 Test Case 1: Unit Testing (Structure-based)

Objective: Test individual units of the system to ensure they work as intended.

Steps:

1. Write test cases for each unit (function/method).
2. Run the test cases on each unit.
3. Record the outcomes for each unit.
4. Update test cases if necessary.

Expected Outcome: All units should pass their respective tests with full coverage.

Actual Outcome: 98% code coverage was achieved. Some edge cases required additional test cases.

11.1.2.2 Test Case 2: Integration Testing (State Transition Testing)

Objective: Ensure the components interact smoothly and transitions between system states are correct.

Steps:

1. Identify system components for integration testing.
2. Develop test cases to check state transitions between components.
3. Run the test cases and record any errors in transitions.
4. Resolve issues identified during testing.

Expected Outcome: State transitions are handled correctly without interrupting the workflow.

Actual Outcome: Some issues with edge cases were found and resolved.

11.1.2.3 Test Case 3: System Testing

Objective: Test the entire system for real-world use cases.

Steps:

1. Create a list of use cases that simulate real-world scenarios.
2. Execute the test cases on the complete system.

3. Observe and document any issues or bugs.
4. Correct bugs and re-run tests to ensure the system works as expected.

Expected Outcome: The system performs seamlessly across all major functionalities.

Actual Outcome: All functionalities were verified, with a few UI bugs noted and fixed.

11.1.3 Experience-Based Testing Techniques

Experience-based techniques rely on the knowledge and intuition of testers. These methods often reveal bugs that structured techniques may not uncover, offering valuable insights from a human perspective.

11.1.3.1 Test Case 1: Error Guessing

Objective: Identify potential bugs based on the testers' experience with similar systems.

Steps:

1. Review the system's input validation processes.
2. Use prior knowledge to identify likely points of failure.
3. Execute tests focused on these failure points.
4. Document any bugs or errors found during testing.

Expected Outcome: Identification of common input validation errors.

Actual Outcome: Two input validation issues were discovered and resolved.

11.1.3.2 Test Case 2: Exploratory Testing

Objective: Identify hidden bugs by freely exploring the system without predefined test cases.

Steps:

1. Start using the system without a structured test plan.
2. Explore different features and workflows in the system.
3. Take notes on any unexpected behaviors or bugs.
4. Report findings and fix any issues encountered.

Expected Outcome: Discover UI bugs and unexpected behaviors.

Actual Outcome: Five UI-related bugs were found and resolved. Navigation improvements were suggested.

11.1.3.3 Test Case 3: Use Case-based Testing

Objective: Test the system based on real-world use case scenarios.

Steps:

1. Identify key user interactions with the system.
2. Develop test cases based on these interactions (e.g., adding items to a shopping cart).
3. Execute the test cases and observe the outcomes.
4. Report and fix any issues encountered.

Expected Outcome: The system works flawlessly for all real-world use cases.

Actual Outcome: All use cases worked as expected, except for a minor session handling issue, which was fixed.

11.1.4 Specification-Based Testing Techniques

Specification-based testing creates test cases from system requirements and specifications to ensure the software meets documented needs.

11.1.4.1 Test Case 1: Equivalence Partitioning

Objective: Validate input handling by dividing inputs into valid and invalid partitions.

Steps:

1. Identify valid and invalid input ranges for different system fields.
2. Select representative values from each partition.
3. Test the system with these inputs.
4. Observe how the system handles both valid and invalid inputs.

Expected Outcome: The system handles inputs correctly, accepting valid inputs and rejecting invalid ones.

Actual Outcome: All valid inputs were accepted, and invalid inputs produced the correct error messages.

11.1.4.2 Test Case 2: Boundary Value Analysis (BVA)

Objective: Ensure the system handles edge cases effectively by testing boundary values.

Steps:

1. Identify the boundary values for each input field.
2. Create test cases that use values at the boundaries.
3. Execute the tests and observe the outcomes.
4. Document and correct any errors.

Expected Outcome: The system correctly handles boundary values.

Actual Outcome: Boundary values were handled as expected, with one issue related to input length validation being corrected.

11.1.4.3 Test Case 3: Decision Table Testing

Objective: Test the system using different combinations of input conditions and expected outcomes.

Steps:

1. Create a decision table mapping different conditions and outcomes.
2. Write test cases based on the table.
3. Execute the test cases, checking for expected outcomes.
4. Document any discrepancies and fix issues.

Expected Outcome: The system produces correct results for all condition combinations.

Actual Outcome: All condition combinations produced expected results, verifying the system's decision-making logic.

11.2 Conclusion

This project applied a comprehensive range of testing techniques—static, dynamic, experience-based, and specification-based—to ensure system quality and reliability. Each test case was structured with clear steps, expected outcomes, and actual results. This rigorous testing process identified key issues and contributed to a more robust, well-functioning system.

Chapter 12

Project Maintenance

This chapter outlines the ongoing maintenance strategies for the SnapShop project to ensure smooth operation, scalability, and updates over time.

12.1 Continuous Integration and Continuous Development

For the SnapShop project, we employ a Continuous Integration (CI) and Continuous Development (CD) approach to make development easy and ensure code reliability. This process is automated through several tools including Docker, Jenkins, GitHub, and others.

- **Docker:** We use Docker to containerize our application, ensuring that it runs consistently across different environments. Each service, such as the frontend, backend, and model, is packaged into containers that can be easily deployed and scaled.
- **Jenkins:** Jenkins serves as our CI/CD automation server. It monitors changes in the code repository, automatically builds the project, runs tests, and deploys the application. With Jenkins, we are able to maintain a steady delivery pipeline that keeps the SnapShop platform up-to-date without manual intervention.
- **GitHub:** GitHub serves as our version control system. All code changes are tracked and reviewed using pull requests. Integration with Jenkins ensures that any code pushed to the repository is tested and built automatically.
- **Other Tools:** In addition to Docker, Jenkins, and GitHub, we also use other tools like Kubernetes for container orchestration, Prometheus for monitoring, and Slack for automated notifications when builds are successful or when issues arise during deployment.

By integrating these tools, we ensure that SnapShop is continuously developed and deployed with minimal downtime, providing seamless updates and improvements to both the backend system and user-facing features.

12.2 Maintenance Test

Regular maintenance tests are carried out to ensure that all SnapShop features function as expected after each update. These include:

- **Unit Testing:** Each component of SnapShop is thoroughly tested for functionality. This includes the image generation model, the data processing pipeline, and the user interface.
- **Integration Testing:** We perform integration tests to verify that the individual components work together as a cohesive system, ensuring that changes made to one service do not negatively impact others.
- **Regression Testing:** After every deployment, regression tests are run to ensure that new updates or changes do not cause previously functioning features to break.
- **Performance Testing:** To ensure that the platform can handle high loads of user traffic and data processing, performance testing is done periodically.

These maintenance tests, in combination with our CI/CD pipeline, ensure that SnapShop remains robust and scalable as new features are added or existing ones are improved.

Chapter 13

Conclusion

This marks end of the First Iteration of the Project. we have covered the Use Cases, Design architectures and Stake Holders and much more in this document. After the First Iteration, everything that has been written revolves around the First Iteration as this has been implemented via code. For the upcoming Iterations, there can be a possibility that the use cases or testing strategies or maintenance might have to further enhanced to keep up with the advanced architectures.

Thank you very much.

Bibliography

- [1] Ar/ai for fashion. page 1, 2023. Accessed: 6 October 2024.
- [2] The fashion industry: An introduction to fashion as an industry. page 1. Bartleby, 2024. Accessed: 10 September 2024.
- [3] Pulpo AR. Pulpo ar - augmented reality solutions, 2020. Accessed: 2024-10-06.
- [4] Yisol Choi, Sangkyung Kwak, Kyungmin Lee, Hyungwon Choi, and Jinwoo Shin. Improving diffusion models for authentic virtual try-on in the wild, 2024.
- [5] Mohsen Gholami, Rabab Ward, and Z Jane Wang. Posegen: Learning to generate 3d human pose dataset with nerf. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 1905–1913, 2024.
- [6] Zero10. Zero10 - ar fashion, 2021. Accessed: 2024-10-06.